GDF15_Energy expenditure_FCaF

Dongdong Wang

4/17/2023

RNAseq analysis in liver tissue

In this section, we share the code about analysis of RNAseq data from liver tissue, please contact us if you have any question about the code

```
library(GEOquery)
library(dplyr)
library(pathview)
library("tximport")
library("readr")
library("tximportData")
library ("AnnotationDbi")
library ("org.Mm.eg.db")
library ("TxDb.Mmusculus.UCSC.mm10.knownGene")

dir <- "D:/R analysisLiver"
samples <- read.table("sample_gdf15liver.txt", header=TRUE)
rownames(samples) <- samples$group

# find the files for each sample, and change the colume name to sample name
files <- file.path(dir,"salmon", samples$run, "quant.sf")
head (files)
names(files) <- samples$group

### Creating the tx2gene data.frame
txdb <- TxDb.Mmusculus.UCSC.mm10.knownGene
keytypes(txdb)
columns(txdb)
k <- keys(txdb, keytype = "GENEID")
table(k)
tx2gene <- select(txdb, keys=k, keytype ="GENEID", columns = "TXNAME")
txi <- tximport(files, type="salmon", tx2gene=tx2gene)

###The ddsTxi object here can then be used as dds in the following analysis steps.
BiocManager::install("DESeq2")
library("DESeq2")
ddsTxi <- DESeqDataSetFromTximport(txi,
                                   colData = samples,
                                   design = ~ condition)
dds<- ddsTxi
dds
```

```
### Here we perform a minimal pre-filtering to keep only rows that have at least 5 reads total.
keep <- rowSums(counts(dds)) >= 5
dds <- dds[keep,]
dds

###PCA
BiocManager::install("pcaExplorer")
library("pcaExplorer")
rld <- rlog(dds, blind=FALSE)
head(assay(rld), 3)
rld
pcaplot(rld,intgroup = "condition",
        pcX = 1, pcY = 4
        , title = "PCA on GDF15-NASH sample RNAseq",
        ellipse = TRUE, text_labels = FALSE)

### Heatmap of sample-to-sample distances using the variance stabilizing transformed values.
sampleDists <- dist(t(assay(vsd)))
library("RColorBrewer")
library("pheatmap")
sampleDistMatrix <- as.matrix(sampleDists)
rownames(sampleDistMatrix) <- vsd$group
colnames(sampleDistMatrix) <- vsd$group
colors <- colorRampPalette( rev(brewer.pal(9, "Blues")) )(255)
pheatmap(sampleDistMatrix,
         clustering_distance_rows=sampleDists,
         clustering_distance_cols=sampleDists,
         col=colors)


####### Step: Differential expression analysis
dds$condition <- relevel(dds$condition, ref = "Vehicle")
dds$condition
dds <- DESeq(dds)
res <- results(dds)
res

#### Contrasts: containing a factor with three levels, say A, B and C
## GDF15 vs Vehicle
res_gvv=results(dds, contrast=c("condition","GDF15","Vehicle"))
res_gvv

## GDF5 vs PairFed
res_gvp=results(dds, contrast=c("condition","GDF15", "PairFed"))
res_gvp
write.csv(as.data.frame(res_gvp), "resGvP_results.csv")


############### Volcano Plot
###GDF15 vs Vehicle
#reset par
par(mfrow=c(1,1))
# Make a basic volcano plot
```

```r
with(res_gvv, plot(log2FoldChange, -log10(pvalue), pch=20, main="GDF15 vs Vehicle (padj<.05&abs(log2FC)>

# Add colored points: blue if padj<0.01, red if log2FC>1 and padj<0.05)
with(subset(res_gvv, padj<.01 ), points(log2FoldChange, -log10(pvalue), pch=20, col="blue"))
with(subset(res, padj<.01 & abs(log2FoldChange)>2), points(log2FoldChange, -log10(pvalue), pch=20, col=

# my setting: padj<.05&abs(log2FC)>1
with(subset(res_gvv, padj<.05 & abs(log2FoldChange)>1), points(log2FoldChange, -log10(pvalue), pch=20, 


###PairFed vs Vehicle
#reset par
par(mfrow=c(1,1))
# Make a basic volcano plot
with(res_pvv, plot(log2FoldChange, -log10(pvalue), pch=20, main="PairFed vs Vehicle (padj<.05&abs(log2F
# my setting: padj<.01&abs(log2FC)>1
with(subset(res_pvv, padj<.05 & abs(log2FoldChange)>1), points(log2FoldChange, -log10(pvalue), pch=20, 

###GDF15 vs PairFed
#reset par
par(mfrow=c(1,1))
# Make a basic volcano plot
with(res_gvp, plot(log2FoldChange, -log10(pvalue), pch=20, main="GDF15 vs PairFed (padj<.05&abs(log2FC)>
# my setting: padj<.01&abs(log2FC)>1
with(subset(res_gvp, padj<.05 & abs(log2FoldChange)>1), points(log2FoldChange, -log10(pvalue), pch=20, 


############################## Enrichment analysis ######################
################## GO enrichment analysis

BiocManager::install("GEOquery")
BiocManager::install("pathview")
BiocManager::install("KEGG.db")

library(GEOquery)
library(dplyr)
library(pathview)
library(enrichplot)
library(clusterProfiler)
library(org.Hs.eg.db)
library(GO.db)
library(KEGG.db)
library(GOstats)
library ("org.Mm.eg.db")
library(Ipaper)

###Prapation dataset for enrichment analysis:log2FC >0.8.5&padj < 0.1
summary(res_gvv, res_gvv$padj < 0.05, res_gvv$log2FoldChange>1.2)

res_gvvOrdered1 <- res_gvv[order(res_gvv$padj),]
res_gvvSig1 <- subset(res_gvvOrdered1, padj < 0.05)
res_gvvSig1
res_gvvOrdered2 <- res_gvvSig1[order(res_gvvSig1$log2FoldChange),]
```

```
res_gvvSig2 <- subset(res_gvvOrdered2, abs(log2FoldChange) >1.2)
res_gvvSig2
write.csv(as.data.frame(res_gvvSig2), "res_gvvPadj0.1logFC0.8_GO KEGG.csv")
res_gvvSig2$SYMBOL=rownames(res_gvvSig2)

### Get dataform
gene <- res_gvvSig2 [,7]
gene.df <- bitr(gene, fromType = "SYMBOL",
                toType = "ENTREZID",
                OrgDb = org.Mm.eg.db)
gene.df

### GO_all enrichment
ego_all <- enrichGO(gene = gene.df$ENTREZID,
                    OrgDb = org.Mm.eg.db,
                    keyType = "ENTREZID",
                    ont = "ALL",
                    pvalueCutoff = 0.01,
                    qvalueCutoff = 0.01)

head(ego_all)
dim(ego_all)
write.csv(ego_all,'GO_ALL_GvV.csv')
dev.off()

dotplot(ego_all, showCategory=30)+ggtitle("GO enrichment analysis")
dotplot1=dotplot(ego_all, showCategory=30)+ggtitle("GO enrichment analysis")
write_fig(dotplot1, "GO enrichment analysis.tif", show = FALSE, devices = "tif", width = 12,
          height = 8)
dev.off()




#################### KEGG pathway enrichment analysis

###Prapation dataset for enrichment analysis:log2FC >0.8&padj < 0.1
summary(res_gvv, res_gvv$padj < 0.05, res_gvv$log2FoldChange>1)

res_gvvOrdered1 <- res_gvv[order(res_gvv$padj),]
res_gvvSig1 <- subset(res_gvvOrdered1, padj < 0.1)
res_gvvSig1
res_gvvOrdered2 <- res_gvvSig1[order(res_gvvSig1$log2FoldChange),]
res_gvvSig2 <- subset(res_gvvOrdered2, abs(log2FoldChange) >0.8)
res_gvvSig2
write.csv(as.data.frame(res_gvvSig2), "res_gvvPadj0.1logFC0.8_GO KEGG.csv")
res_gvvSig2$SYMBOL=rownames(res_gvvSig2)

### Get dataform
gene <- res_gvvSig2 [,7]
gene.df <- bitr(gene, fromType = "SYMBOL",
                toType = "ENTREZID",
                OrgDb = org.Mm.eg.db)
```

```
gene.df

#get Universal background gene
res_gvvall=res_gvv
res_gvvall$SYMBOL=rownames(res_gvvall)
gene_all <- res_gvvall [,7]
gene.all1 <- bitr(gene_all, fromType = "SYMBOL",
                  toType = "ENTREZID",
                  OrgDb = org.Mm.eg.db)
gene.all1

### KK enrichment
kk <- enrichKEGG(gene = gene.df$ENTREZID,
                 organism = 'mmu',
                 keyType = "kegg",
                 universe= gene.all1$ENTREZID,
                 pvalueCutoff = 0.05,
                 qvalueCutoff = 0.05)
head(kk)[,1:6]
dim(kk)
write.csv(kk,'KEGG_GvV.csv')
dev.off()
dotplot(kk, showCategory=30)+ggtitle("KEGG pathway enrichment analysis")
dotplot1=dotplot(kk, showCategory=30)+ggtitle("KEGG pathway enrichment analysis")
write_fig(dotplot1, "6_KEGG pathway enrichment analysis.tif", show = FALSE, devices = "tif", width = 12
          height = 8)
dev.off()




############ cnetplot for GO log2FC >2&padj < 0.01
###Prapation dataset for enrichment analysis:log2FC >1.5&padj < 0.05
summary(res_gvv, res_gvv$padj < 0.05, res_gvv$log2FoldChange>1)

res_gvvOrdered1 <- res_gvv[order(res_gvv$padj),]
res_gvvSig1 <- subset(res_gvvOrdered1, padj < 0.05)
res_gvvSig1
res_gvvOrdered2 <- res_gvvSig1[order(res_gvvSig1$log2FoldChange),]
res_gvvSig2 <- subset(res_gvvOrdered2, abs(log2FoldChange) >1.5)
res_gvvSig2
write.csv(as.data.frame(res_gvvSig2), "res_gvvPadj0.01logFC2_Circle.csv")
res_gvvSig2$SYMBOL=rownames(res_gvvSig2)

### Get dataform
BiocManager::install("ggnewscale")
library(ggnewscale)

gene <- res_gvvSig2 [,7]
gene.df <- bitr(gene, fromType = "SYMBOL",
                toType = "ENTREZID",
                OrgDb = org.Mm.eg.db)
```

```
gene.df

ego_all <- enrichGO(gene = gene.df$ENTREZID,
                    OrgDb = org.Mm.eg.db,
                    keyType = "ENTREZID",
                    ont = "all",
                    pvalueCutoff = 0.05,
                    qvalueCutoff = 0.05)


res_gvvSig2=as.data.frame(res_gvvSig2)
res_gvvSig2mg=merge(res_gvvSig2, gene.df, by="SYMBOL")
res_gvvSig2mg
geneList = res_gvvSig2mg[,3]
head(geneList)
names(geneList) = as.character(res_gvvSig2mg[,1])
geneList = sort(geneList, decreasing = TRUE)

oragnx <- setReadable(ego_all, 'org.Mm.eg.db', 'ENTREZID')
cnetplot(oragnx, foldChange=geneList,
         circular = TRUE,
         colorEdge = TRUE)
fig1=cnetplot(oragnx, foldChange=geneList,
              circular = TRUE,
              colorEdge = TRUE)
write_fig(fig1, "5_GO_circle.tif", show = FALSE, devices = "tif", width = 15,
          height = 8)
dev.off()


############### Heatmap of human 25-gene signature (PMID: 33268509).bad results

### output count data.
unnormalized_counts <- counts(dds, normalized = F)
head(unnormalized_counts)
class(unnormalized_counts)
write.csv(data.frame(unnormalized_counts), '1-counts.csv')

unnor_counts=data.frame(unnormalized_counts)
head(unnor_counts)
class (unnor_counts)
unnor_counts$symbol=row.names(unnor_counts)
genenam25=read.csv('25genesig.csv', header = T)
genenam25
row.names(genenam25)=genenam25$symbol
genenam25

unnor_counts25=merge(unnor_counts, genenam25, by="symbol")
unnor_counts25=unnor_counts25[,-20]
row.names(unnor_counts25)=unnor_counts25$symbol
unnor_counts25
unnor_counts25=unnor_counts25[,-1]
a1=unnor_counts25
```

```
colnames(unnor_counts25)

##heatmap
a1=as.matrix(a1)

dat=a1
dim(dat)
dat
dat=dat[-19,]
dim(dat)
dat
dat=dat[-11,]
dat
dim(dat)
dat=dat[-2,]
dat
dim(dat)
dat=dat[-6,]
dat
dim(dat)

#n=vst(dat, blind = TRUE, nsub = 21, fitType = "parametric")
dat

dat=log2(dat+1)
n=t(scale(t(dat)))

library(pheatmap)
n[n>2]=2
n[n<-2]=-2
n[1:4,1:4]
pheatmap(n, show_colnames = T,cluster_cols = T,
         show_rownames = T, cluster_rows = T, fontsize_row = 9, fontsize_col=9)
pheatmap1=pheatmap(n, show_colnames = T,cluster_cols = T,
                   show_rownames = T, cluster_rows = T)
?pheatmap
library(Ipaper)
write_fig(pheatmap1, "8_25genesig_heatmap.tif", show = FALSE, devices = "tif", width = 9,
          height = 9)



############## Heatmap of Nanostring fibrosis-gene signature
### output count data.
unnormalized_counts <- counts(dds, normalized = F)
head(unnormalized_counts)
class(unnormalized_counts)
write.csv(data.frame(unnormalized_counts), '1-counts.csv')

unnor_counts=data.frame(unnormalized_counts)
head(unnor_counts)
class (unnor_counts)
unnor_counts$symbol=row.names(unnor_counts)
```

```
genesigNano=read.csv('Nanostring_Fibrosis panel.csv', header = T)
genesigNano
row.names(genesigNano)=genesigNano$symbol
genesigNano
genenam25=genesigNano

unnor_counts25=merge(unnor_counts, genenam25, by="symbol")
unnor_counts25
unnor_counts25=unnor_counts25[,-20]
row.names(unnor_counts25)=unnor_counts25$symbol
unnor_counts25
unnor_counts25=unnor_counts25[,-1]
a1=unnor_counts25
colnames(unnor_counts25)

##heatmap
a1=as.matrix(a1)

dat=a1
dim(dat)
dat

#n=vst(dat, blind = TRUE, nsub = 21, fitType = "parametric")

cg=names(tail(sort(apply(dat,1,sd)), 70))
#write.csv(cg,'Nanostring_sd50heatmap.csv')
#library(pheatmap)
dat=log2(dat+1)
n=t(scale(t(dat[cg,])))
#n[n>2]=2
#n[n<-2]=-2
#n[1:4,1:4]
pheatmap(n, show_colnames = T,
         show_rownames = T)
pheatmap1=pheatmap(n, show_colnames = T,
                   show_rownames = T)

write_fig(pheatmap1, "7_Nanogenesig_heatmap.tif", show = FALSE, devices = "tif", width = 6,
          height = 12)


########################PCA in protein kinase A signaling in liver (GO:0010737)

library(GEOquery)
library(dplyr)
library(pathview)
library(Ipaper)
library(ggplot2)
library(reshape2)
library(ggpubr)
library(ggstatsplot)
library(tidyverse)
library(dplyr)
```

```
library(tidyr)
library(stringr)
library( "DESeq2" )


#input count data and adrenergic signaling.
a=read.csv('GO0010737.csv', header = TRUE)

b=read.csv('1-counts.csv', header = TRUE)
b$Symbol=b$X

c=merge(a, b, by="Symbol")

write.csv(c, 'GO_PKA counts.csv')
## delete other columns except "Symbol, X"
c=read.csv('GO_PKA counts.csv', header = TRUE)
rownames(c)=c$Symbol
c1=c[,c(-1,-2,-3)]

######### DESeq2 analysis
countData=c1

samples <- read.table("sample_gdf15liver.txt", header=TRUE)
head (samples)
metaData=samples

####### Construct DESEQDataSet Object
library("DESeq2")

dds <- DESeqDataSetFromMatrix(countData=countData,
                              colData=metaData,
                              design = ~ condition, tidy = F)

dds


###PCA
##First we need to transform the raw count data
## Data transformations and visualization
##for visualization or clustering - it might be useful to work with transformed versions of the count d
## rlog function might take too long, variance stabilizing transformations (VST) function will be a fas
## By setting blind to FALSE
## vst function will perform variance stabilizing transformation
vsd <- vst(dds, blind=FALSE)
head(assay(vsd), 3)

vsd1=assay(vsd)

write.csv(as.data.frame(vsd1), "1-counts_Mus2Mus.csv")
#using the DESEQ2 plotPCA fxn we can
plotPCA(vsd, intgroup="Groups")

#label the sample names
```

```
library(ggplot2)
plotPCA(vsd) + geom_text(aes(label=name),vjust=0.25)

###PCA2
BiocManager::install("pcaExplorer")
library("pcaExplorer")
rld <- rlog(dds, blind=FALSE)
head(assay(rld), 3)
rld
pcaplot(rld,intgroup = "condition",
        pcX = 1, pcY = 3, title = "PCA on liver RNAseq based on GO:0010737",
        ellipse = T, text_labels = FALSE)

pheatmap1=pcaplot(rld,intgroup = "condition",
                  pcX = 1, pcY = 3, title = "PCA on liver RNAseq based on GO:0010737",
                  ellipse = T, text_labels = FALSE)

library(Ipaper)
write_fig(pheatmap1, "PCA on liver RNAseq based on GO 0010737.tif", show = FALSE, devices = "tif", widt
          height = 4)
```

RNAseq analysis in TA muscle

In this section, we share the code about analysis of RNAseq data from in TA muscle, please contact us if you have any question about the code.

```
library(GEOquery)
library(dplyr)
library(pathview)
library("tximport")
library("readr")
library("tximportData")
library ("AnnotationDbi")
library ("org.Mm.eg.db")
library ("TxDb.Mmusculus.UCSC.mm10.knownGene")


dir <- "D:/R analysisTA"
samples <- read.table("sample_infoWTGDF15.txt", header=TRUE)

# construct condition
rownames(samples) <- samples$group
# find the files for each sample, and change the colume name to sample name
files <- file.path(dir,"salmon", samples$run, "quant.sf")
head (files)
names(files) <- samples$group


### Creating this tx2gene data.frame can be accomplished from a TxDb object and the select function fro
txdb <- TxDb.Mmusculus.UCSC.mm10.knownGene
keytypes(txdb)
columns(txdb)
```

```
k <- keys(txdb, keytype = "GENEID")
table(k)
tx2gene <- select(txdb, keys=k, keytype ="GENEID", columns = "TXNAME")


### We import the necessary quantification data for DESeq2
tx2gene <- read.csv("tx2geneUCSCmm10.csv", header=TRUE)
head (tx2gene)
txi <- tximport(files, type="salmon", tx2gene=tx2gene)

### Finally, we can construct a DESeqDataSet from the txi object
BiocManager::install("DESeq2")
library("DESeq2")
ddsTxi <- DESeqDataSetFromTximport(txi,
                                   colData = samples,
                                   design = ~ condition)
dds<- ddsTxi
dds

### Here we perform a minimal pre-filtering to keep only rows that have at least 10 reads total.
keep <- rowSums(counts(dds)) >= 5
dds <- dds[keep,]
dds

### Heatmap of sample-to-sample distances using the Poisson Distance.
library("PoiClaClu")
library("pheatmap")
poisd <- PoissonDistance(t(counts(dds)))
samplePoisDistMatrix <- as.matrix( poisd$dd)
rownames(samplePoisDistMatrix) <- dds$group
colnames(samplePoisDistMatrix) <- dds$group
pheatmap(samplePoisDistMatrix,
         clustering_distance_rows = poisd$dd,
         clustering_distance_cols = poisd$dd
         )
library(Ipaper)
pheatmap1=pheatmap(samplePoisDistMatrix,
                   clustering_distance_rows = poisd$dd,
                   clustering_distance_cols = poisd$dd
)
write_fig(pheatmap1, "heatmap_sample2sample distance.tif", show = FALSE, devices = "tif", width = 5,
          height = 4)
```

Mendelian randomization (MR) study

In this section, we share the code about MR study about GDF15 and liver fat/volume, please contact us if you have any question about the code.

```
install.packages("remotes")
remotes::install_github("MRCIEU/TwoSampleMR")
devtools::install_github("MRCIEU/MRInstruments")
install.packages('data.table')
```

```r
install.packages("knitr")

library(TwoSampleMR)
library(MRInstruments)
library(data.table)
library(knitr)
library(ggplot2)
library(Ipaper)


############GDF15-liver fat_"ebi-a-GCST90011998" (GDF15) and "ebi-a-GCST90016673" (liver fat)

## List available GWASs
ao <- available_outcomes()
# Get instruments:Extracting particular SNPs from particular studies, "prot-a-1195" (from Nature) nsnp:
# and Clump
bmi_exp_dat <- extract_instruments("ebi-a-GCST90011998", p1=5e-08,clump=F)
dim(bmi_exp_dat)
write.csv(bmi_exp_dat, paste0("gdf15-LF-exp bf clump", ".csv"))
#238 snps

# Clump
bmi_exp_dat <- extract_instruments("ebi-a-GCST90011998", p1=5e-08,clump=TRUE, p2 = 1e-07, r2=0.001, kb=
dim(bmi_exp_dat)
write.csv(bmi_exp_dat, paste0("gdf15-LF-exp af clump", ".csv"))

# The extract_outcome_data() function is flexible.
# The snps argument only requires an array of rsIDs, and the outcomes argument can be a vector of outco
# The most recent 'percent liver fat' GWAS is ID number "ebi-a-GCST90016673". We can extract the SNPs f

chd_out_dat  <- extract_outcome_data(snps = bmi_exp_dat$SNP, outcomes='ebi-a-GCST90016673')

write.csv(chd_out_dat, paste0("gdf15-LF-outcome", ".csv"))

# Harmonise the exposure and outcome data
dat <- harmonise_data(
  exposure_dat = bmi_exp_dat,
  outcome_dat = chd_out_dat)
write.csv(dat, paste0("gdf15-LF-harmonise_1", ".csv"))

# Drop duplicate exposure-outcome summary sets
dat<-power_prune(dat,method=1,dist.outcome="binary")


## Sensitivity analyses
# Heterogeneity statistics
het <-mr_heterogeneity(dat)
het
write.csv(het, paste0("gdf15-LF-Heterogeneity_1", ".csv"))

#If it is Heterogeneity, then we use the model "mr_ivw_mre" which is Inverse variance weighted "(multip
res=mr(dat,method_list=c('mr_ivw_mre'))
res
```

```r
#Horizontal pleiotropy
pleio <-mr_pleiotropy_test(dat)
pleio
write.csv(pleio, paste0("gdf15-LF-pleiotropy_1", ".csv"))

#Leave-one-out analysis
res_loo <- mr_leaveoneout(dat)
mr_leaveoneout_plot(res_loo)
pheatmap1=mr_leaveoneout_plot(res_loo)
write.csv(res_loo, paste0("gdf15-LF-Leave-one-out_1", ".csv"))

library(Ipaper)
write_fig(pheatmap1, "gdf15-LF-Leave-one-out.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 2)



# Perform MR
res <- mr(dat)
res
write.csv(res, paste0("gdf15-LF-MR_1", ".csv"))


#To obtain the MR estimates using each of the SNPs singly we can do the following:
res_single <- mr_singlesnp(dat)
write.csv(res_single, paste0("gdf15-LV-MR-single snp_1", ".csv"))


# MR Steiger directionality test: test causal direction
out <- directionality_test(dat)
out
write.csv(out, paste0("gdf15-LF-MR-causal dire_1", ".csv"))


# Scatter plot
p1 <- mr_scatter_plot(res, dat)
p1[[1]]

#ggsave(p1[[1]], file="gdf15-BMR-MR scatter_1.png", width=10, height=12, units ="cm")

pheatmap1=p1[[1]]
write_fig(pheatmap1, "gdf15-LF-MR scatter_1.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 5)


# Forest plot
res_single <- mr_singlesnp(dat)
p2 <- mr_forest_plot(res_single)
p2[[1]]

#ggsave(p2[[1]], file="gdf15-BMR-MR forest_1.png", width=10, height=10, units ="cm")

pheatmap1=p2[[1]]
```

```r
write_fig(pheatmap1, "gdf15-LF-MR forest_1.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 2)



# Funnel plot
res_single <- mr_singlesnp(dat)
p4 <- mr_funnel_plot(res_single)
p4[[1]]
#ggsave(p4[[1]], file="gdf15-BMR-MR Funnel_1.png", width=10, height=10, units ="cm")

pheatmap1=p4[[1]]
write_fig(pheatmap1, "gdf15-LF-MR Funnel_1.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 5)









###########GDF15-liver fat_"ebi-a-GCST90011998" (GDF15) and "ebi-a-GCST90016666" (liver volume)

## List available GWASs
ao <- available_outcomes()
# Get instruments:Extracting particular SNPs from particular studies, "prot-a-1195" (from Nature) nsnp:
# and Clump
bmi_exp_dat <- extract_instruments("ebi-a-GCST90011998", p1=5e-08,clump=F)
dim(bmi_exp_dat)
write.csv(bmi_exp_dat, paste0("gdf15-LF-exp bf clump", ".csv"))
#238 snps

# Clump
bmi_exp_dat <- extract_instruments("ebi-a-GCST90011998", p1=5e-08,clump=TRUE, p2 = 1e-07, r2=0.001, kb=
dim(bmi_exp_dat)
write.csv(bmi_exp_dat, paste0("gdf15-LF-exp af clump", ".csv"))

# The extract_outcome_data() function is flexible.
# The snps argument only requires an array of rsIDs, and the outcomes argument can be a vector of outco
# The most recent 'percent liver fat' GWAS is ID number "ebi-a-GCST90016666". We can extract the SNPs f

chd_out_dat  <- extract_outcome_data(snps = bmi_exp_dat$SNP, outcomes='ebi-a-GCST90016666')

write.csv(chd_out_dat, paste0("gdf15-LV-outcome", ".csv"))

# Harmonise the exposure and outcome data
dat <- harmonise_data(
  exposure_dat = bmi_exp_dat,
  outcome_dat = chd_out_dat)
write.csv(dat, paste0("gdf15-LV-harmonise_1", ".csv"))

# Drop duplicate exposure-outcome summary sets
dat<-power_prune(dat,method=1,dist.outcome="binary")
```

```r
## Sensitivity analyses
# Heterogeneity statistics
het <-mr_heterogeneity(dat)
het
write.csv(het, paste0("gdf15-LF-Heterogeneity_1", ".csv"))

#If it is Heterogeneity, then we use the model "mr_ivw_mre" which is Inverse variance weighted "(multip
res=mr(dat,method_list=c('mr_ivw_mre'))
res

#Horizontal pleiotropy
pleio <-mr_pleiotropy_test(dat)
pleio
write.csv(pleio, paste0("gdf15-LF-pleiotropy_1", ".csv"))

#Leave-one-out analysis
res_loo <- mr_leaveoneout(dat)
mr_leaveoneout_plot(res_loo)
pheatmap1=mr_leaveoneout_plot(res_loo)
write.csv(res_loo, paste0("gdf15-LF-Leave-one-out_1", ".csv"))

library(Ipaper)
write_fig(pheatmap1, "gdf15-LF-Leave-one-out.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 2)



# Perform MR
res <- mr(dat)
res
write.csv(res, paste0("gdf15-LV-MR_1", ".csv"))


#To obtain the MR estimates using each of the SNPs singly we can do the following:
res_single <- mr_singlesnp(dat)
write.csv(res_single, paste0("gdf15-LV-MR-single snp_1", ".csv"))


# MR Steiger directionality test: test causal direction
out <- directionality_test(dat)
out
write.csv(out, paste0("gdf15-LF-MR-causal dire_1", ".csv"))


# Scatter plot
p1 <- mr_scatter_plot(res, dat)
p1[[1]]

#ggsave(p1[[1]], file="gdf15-BV-MR scatter_1.png", width=10, height=12, units ="cm")

pheatmap1=p1[[1]]
write_fig(pheatmap1, "gdf15-LV-MR scatter_1.tif", show = FALSE, devices = "tif", width = 6.5,
```

```
          height = 5)


# Forest plot
res_single <- mr_singlesnp(dat)
p2 <- mr_forest_plot(res_single)
p2[[1]]

#ggsave(p2[[1]], file="gdf15-BMR-MR forest_1.png", width=10, height=10, units ="cm")

pheatmap1=p2[[1]]
write_fig(pheatmap1, "gdf15-LF-MR forest_1.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 2)



# Funnel plot
res_single <- mr_singlesnp(dat)
p4 <- mr_funnel_plot(res_single)
p4[[1]]
#ggsave(p4[[1]], file="gdf15-BMR-MR Funnel_1.png", width=10, height=10, units ="cm")

pheatmap1=p4[[1]]
write_fig(pheatmap1, "gdf15-LF-MR Funnel_1.tif", show = FALSE, devices = "tif", width = 6.5,
          height = 5)
```